

CHAPTER THREE

METHODOLOGY AND SYSTEM ANALYSIS

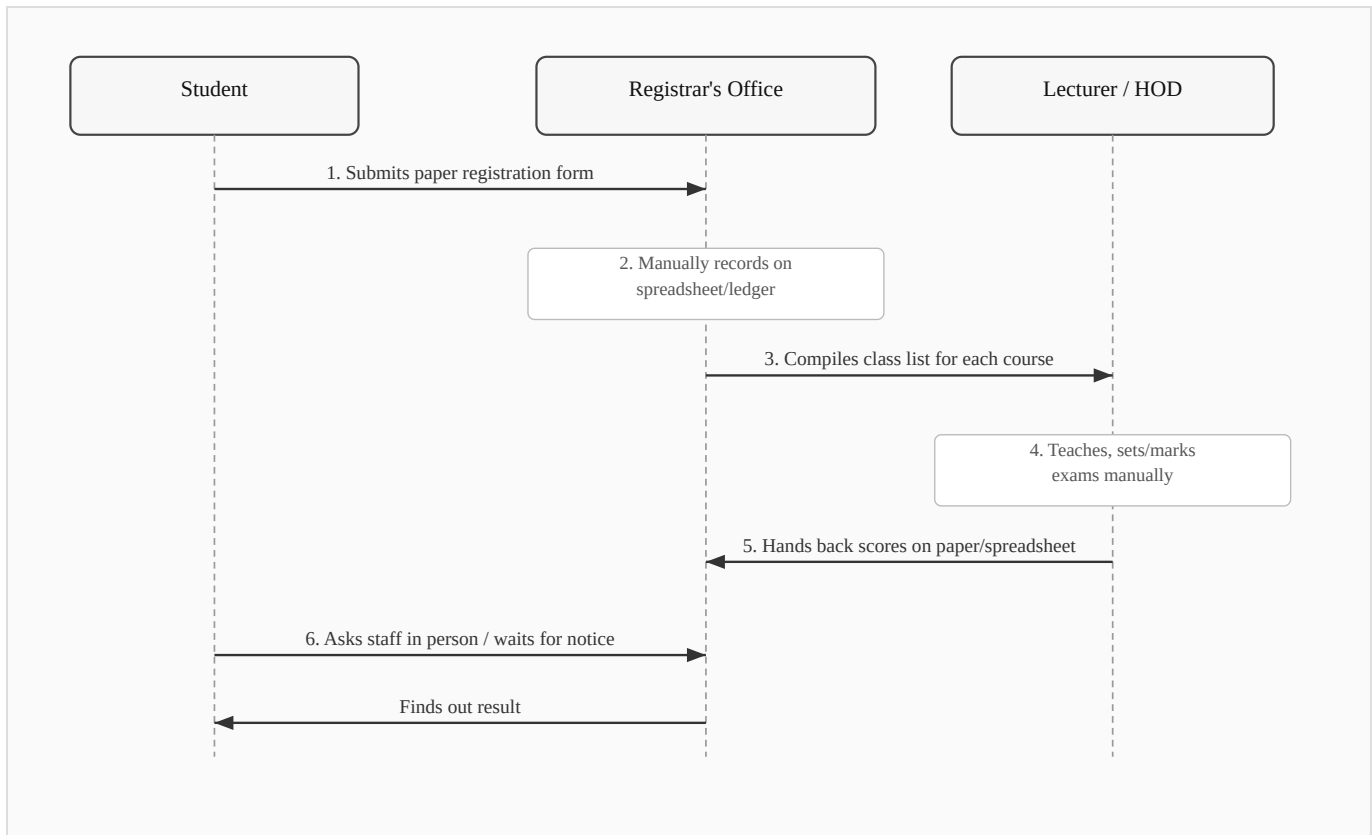
3.1 System Analysis

System analysis, here, means two things done in sequence: first understanding how Legacy University currently handles the processes this project touches, well enough to say precisely what's wrong with it, and second working out what a replacement needs to do to actually fix those problems rather than just moving them onto a screen. The two sections below cover each in turn.

3.1.2 Analysis of the Existing System

Legacy University's current process for course registration, results, and student record-keeping is entirely manual, built around paper forms, spreadsheets kept by individual staff members, and in-person coordination between students, lecturers, and the registrar's office. There is no single system of record; the closest thing to one is whatever spreadsheet a given staff member last updated.

Figure 3.1: Dataflow of the Existing (Manual) System (*sketch below; to be redrawn as a formal DFD for final submission*)



The flow above has no built-in access control. Anyone with access to a given spreadsheet can, in principle, open and edit it regardless of whether they have any legitimate role in that particular course or department, and there is no mechanism forcing a change to stay within the boundary of who made it or why.

3.1.3 Weaknesses of the Existing System

The manual system's problems fall into a few clear categories. Access isn't bounded in any reliable way: nothing stops a staff member from opening a record that has nothing to do with their role, and nothing logs it if they do. Nor is there a single source of truth; the same piece of information, a student's registered courses, for instance, can exist in slightly different states across two or three different spreadsheets, and reconciling them after the fact is tedious and error-prone. Turnaround is slow too, since almost every step depends on a specific person being available to process a form or answer a question in person. And there's no audit trail: if a result is entered wrong, or a registration disappears, there's no record showing when it happened, who touched it, or how to reverse it. None of these are unusual problems for a manual system to have; they're exactly the class of problem RBAC and a centralized database are meant to solve, which is the case this project is built to make.

3.2.1 Methodology Adopted

This project follows an **object-oriented, iterative and incremental** development approach, built and tested one working slice at a time rather than designed exhaustively up front and implemented in a single pass. That choice was less a stylistic preference than a practical necessity: the original design, discussed in Chapter One and revisited in Section 3.2.4 below, specified a broader scope than a single developer could deliver on a final year project's timeline, and it only became clear which parts of that scope were realistic to keep once the earlier increments were built, tested, and evaluated against what remained of the schedule.

Concretely, that meant building authentication and the role decorators first, since every other module depends on them; then student and department records; then course registration; then results and grading; and finally a dedicated security-hardening pass once the functional modules were in place. Each increment came with its own automated test coverage, using Django's `TestCase` framework, rather than being verified by hand and left untested, so that later changes could be checked against earlier behaviour rather than re-verified manually every time. Git version control tracked each increment as its own set of commits, which is also how the scope changes documented in Section 3.2.4 and in the accompanying SRS/SDD can be traced concretely rather than just asserted.

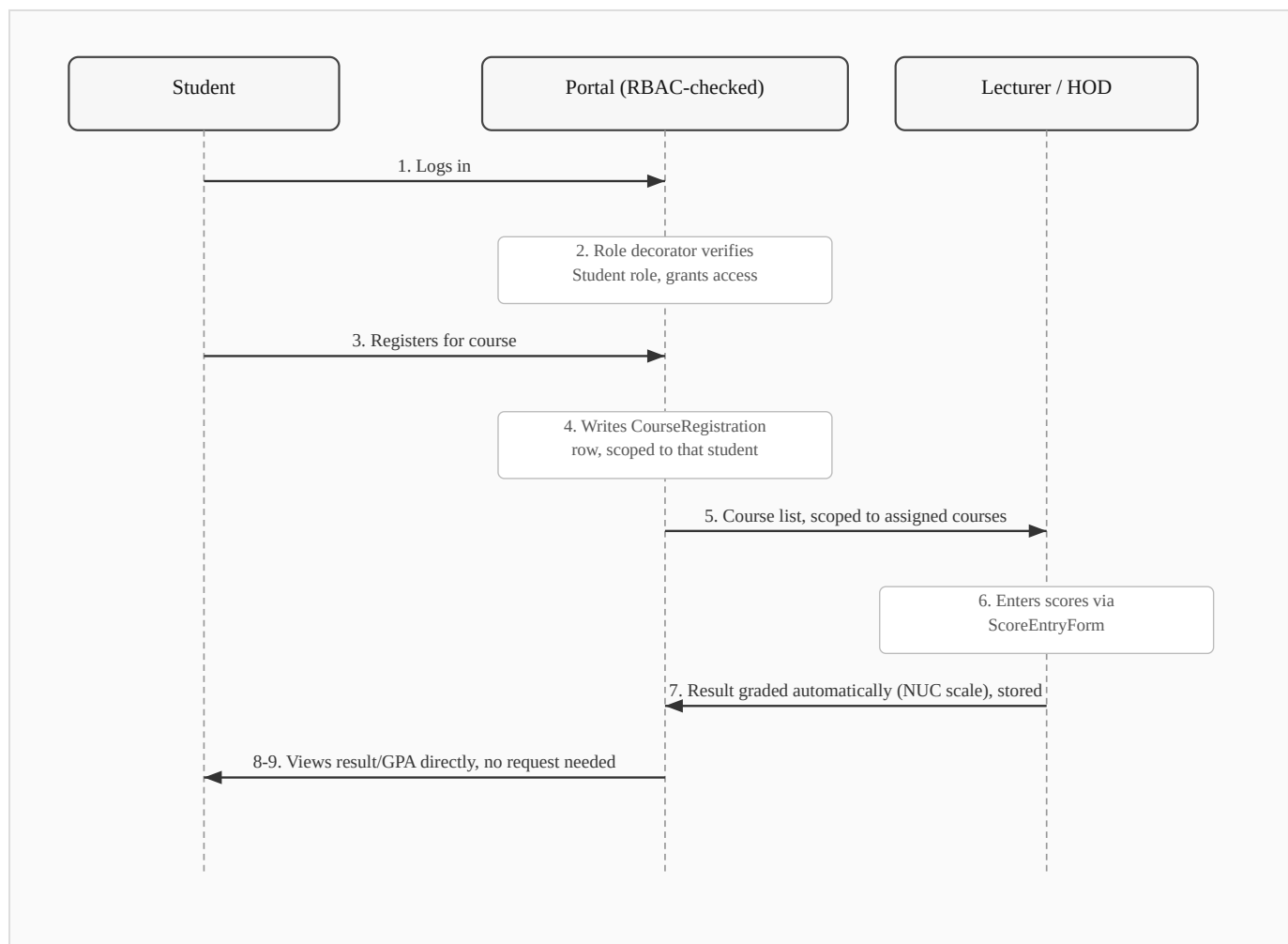
The object-oriented half of that description reflects Django's own architecture more than a deliberate methodological choice: every entity in the system, a user, a course, a result, is modelled as a class with its own fields and behaviour, and the relationships between them (a student has one department, a course belongs to one department, a result belongs to one registration) are expressed as object relationships enforced by the ORM rather than as loose foreign keys managed by hand.

3.2.2 Analysis of the Proposed System

The proposed system replaces the manual flow in Section 3.1.2 with a single Django application backed by a role-based access control layer. Four modules make up the system as built: authentication and RBAC, student information, course registration, and results/grading, covering seven roles (Student, Lecturer, HOD, Registrar, Bursar, Dean, IT Admin) as detailed in the SRS.

Every request that touches protected data passes through a role check before any view logic runs, which closes the access-boundary gap identified in Section 3.1.3 directly: a Lecturer's account is mechanically incapable of editing a course they aren't assigned to, not because of a convention staff are expected to follow, but because the decorator guarding that view checks for it on every request. Student records, course data, and results all live in one database rather than scattered spreadsheets, so there's exactly one place a given fact about a student or a course can be found, and results are graded automatically against the NUC's five-point scale the moment a lecturer enters a score, removing a step that used to depend on someone doing the arithmetic by hand.

Figure 3.2: Dataflow Diagram of the Web-Based (Proposed) System *(sketch below; to be redrawn as a formal DFD for final submission)*



Compared with Figure 3.1, the same underlying steps happen (register, teach, grade, publish), but every arrow in this version passes through a role check before it's allowed to touch the database, and step 8, checking a result, needs no human intermediary at all.

3.2.3 Overall Use Case Diagram of the New System

Figure 3.3: Overall Use Case Diagram of the New System *(the formal UML diagram is drawn separately for final submission; the actor/use-case breakdown below is its content in tabular form, and matches Section 7 of the SRS)*

Actor	Primary use cases
Student	Log in (password or first-login PIN); view dashboard; register for courses; view results and GPA/CGPA; manage own account (preferred username, password, email)
Lecturer	Log in; view assigned courses; enter and update results
HOD	Log in; manage departmental courses
Registrar	Log in; register students individually or via bulk import; edit student profiles; search student records
Dean	Log in; view courses across faculty departments (read-only)
Bursar	Log in; view dashboard (no further functionality implemented; see Section 3.2.4 and Chapter One, Section 1.5)
IT Admin	Log in; manage staff accounts; manage faculties and departments

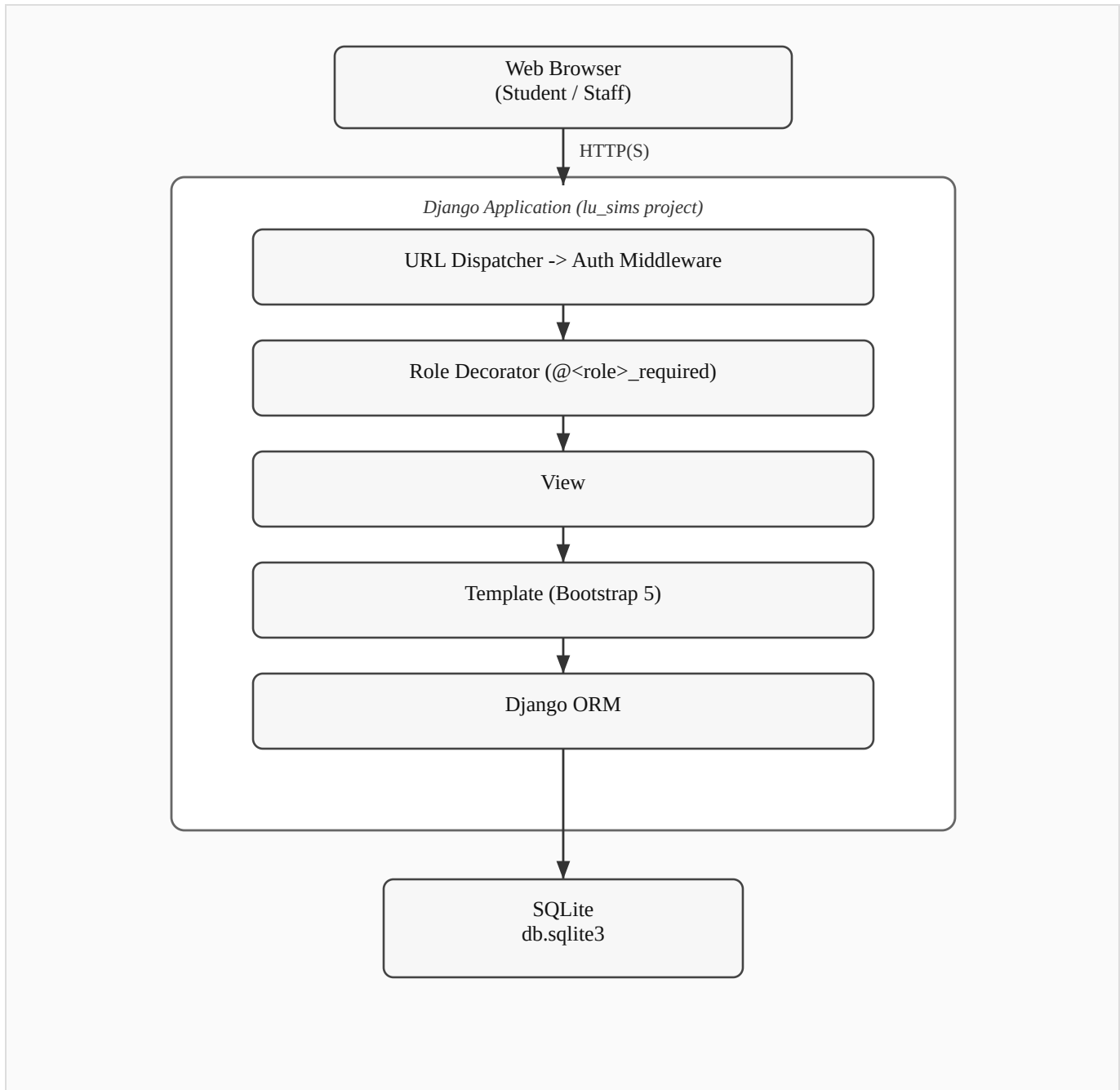
3.2.4 Justification of the New System

The case for replacing the manual process isn't just that a computer is faster than a filing cabinet; it's that RBAC, specifically, closes the access-control gap that a plain digitized spreadsheet wouldn't. A spreadsheet uploaded to a shared drive is still a spreadsheet: anyone with the link can open it, and nothing about the format stops them from editing a field that isn't theirs to touch. Building the same data into a system where every view is guarded by a role check moves that boundary from a social convention ("please don't edit records outside your department") to something the application itself enforces on every request.

The narrower scope actually delivered, four modules instead of the nine originally planned, is a direct consequence of the iterative methodology in Section 3.2.1: rather than attempting the full seven-role, twenty-one-permission design and risking an unfinished system in every module, effort concentrated on a smaller set of modules built to the point of being genuinely usable. Section 1.5 and Section 5.2 of the SRS document exactly what was cut and why. The Results/Grading module in particular replaced the originally-planned Attendance module, since it demonstrates the RBAC model across three roles at once (Student viewing, Lecturer entering, HOD's department scoping the course) rather than the single-role read/write pattern attendance tracking would have involved, making it the stronger choice for a project meant to showcase role-based access control specifically.

3.3.3 High Level Model of the New System

Figure 4.1: High Level Model of the New System (numbered per the department's List of Figures, which sequences this diagram under Chapter Four even though the section itself is 3.3.3; sketch below, to be redrawn as a formal architecture diagram for final submission)



A request reaches the URL dispatcher first, then Django's own authentication middleware, which redirects anyone unauthenticated straight to the login page. Past that point, the role decorator on the matching view checks the current user's role before any business logic executes; if the check fails, the request is denied with a 403 rather than being allowed to fall through to the view. Only once both checks pass does the view run, read or write through the ORM, and render a template back to the browser. This is the same request

flow described in more technical detail in Chapter Four and in Section 2 of the accompanying SDD; this figure gives the same picture at the level System Analysis needs, without the implementation specifics that belong in the design chapter.